University of Engineering and Technology Lahore

Department of Electrical Engineering

Computer Architecture Lab Manual

Course Code: EE-475L

Spring 2026

Contents

Introduction **2**

Experiment 1: Design of Arithmetic Logic Unit and its Controllers **3**
1.1 Overview . . . 3
1.2 ALU . . . 3
1.2.1 Block Diagram . . . 3
1.2.2 Understanding the RISCV ISA Cheat Sheet (Reference Manual) . 4
1.3 Controller Design . . . 4
1.4 Deliverables . . . 6

Experiment 2: Implementation of the ALU and its Controller in SystemVerilog **8**
2.1 Overview . . . 8
2.2 ALU Implementation in SystemVerilog . . . 8
2.3 ALU Controller Implementation . . . 9
2.3.1 Use of `opcode.vh` File . . . 9
2.4 Deliverables . . . 10

Experiment 3: Implementation of Immediate Generator in SystemVerilog **11**
3.1 Overview . . . 11
3.2 Immediate Generator . . . 11
3.2.1 Block Diagram . . . 11
3.2.2 Implementation . . . 11
3.2.3 Testing . . . 12
3.3 Deliverables . . . 12

Introduction

The goal of this lab is to familiarize students with the process of designing a RISC-V processor, beginning with a single-cycle implementation and then extending it into a pipelined architecture capable of handling control and data hazards. This course provides practical exposure to designing digital systems using RTL descriptions, resolving hazards in a simple pipeline, and building functional interfaces. Additionally, students will learn how to approach system-level optimization, bridging the gap between individual components and a fully integrated processor.

In tackling these challenges, your first step will be to map the high-level specification to a design that can be translated into a hardware implementation. Following that, you will produce and debug the implementation. These steps can take a significant amount of time if the design is not carefully planned before implementation.

By the end of this lab, students will have gained hands-on experience in processor design, hazard resolution, and system-level thinking, equipping them with essential skills for real-world digital system development. **We will basically follow this EECS151_SP24 document created by University of Berkeley to design the pipelined processor.**

Experiment 1

Design of Arithmetic Logic Unit and its Controllers

1.1 Overview

In this experiment, students will design and implement an ALU and its controller using SystemVerilog. The design will support RISC-V instructions and will be verified through simulation. This experiment serves as a foundational step toward building a complete RISC-V processor, as the ALU and its controller are critical components used throughout the datapath in later experiments.

1.2 ALU

The Arithmetic Logic Unit (ALU) is a fundamental component of a processor responsible for performing arithmetic and logical operations. These operations include addition, subtraction, logical AND/OR/XOR, comparison, and shift operations.

1.2.1 Block Diagram

Task 1: Draw the schematic / block diagram of the ALU shown in Figure 2.1 using draw.io.

The input and output port names are as shown in Figure 2.1. The ALU module takes two input operands and performs an operation specified by the control signal **alu_operation** and produces the **result**. In addition to the result, the ALU generates a **zero** flag, which is asserted when the result of a subtraction operation is zero.

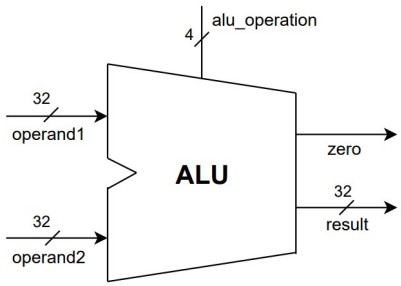

Figure 1.1: ALU

1.2.2 Understanding the RISCV ISA Cheat Sheet (Reference Manual)

Read and understand the first page of this Riscv Cheat Sheet. For more details you can refer to this reference manual.

1.3 Controller Design

The ALU controller determines which operation the ALU should perform for a given instruction. It takes different fields of the RISCV instruction such as **OPCODE, func3** etc. and translates them into a specific control code that drives the ALU. Before designing the controller, you should decide the names and size of the inputs and outputs of the controller. Figure 2.2 shows the controller you will be implementing. The output **alu_operation** depends on three inputs **func3**, **func7** and **ALUOp**.

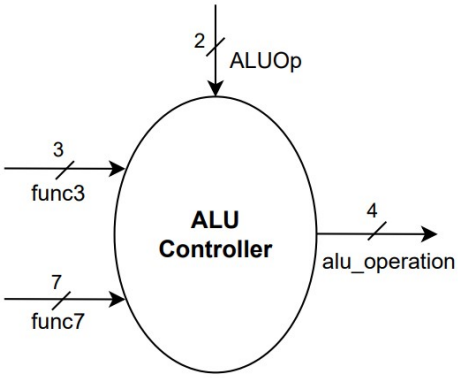

Figure 1.2: ALU Controller

Task 2: Look at the ISA cheat sheet and list down all the arithmetic, bitwise and shift operations in the first column of table 1.1, to be performed by the ALU. The operators must have at least one of `rs1, rs2 or imm` as one of the operands. Decide the values and size of the output **alu_operation** in binary yourself. Use WORD or EXCEL to complete the table. The values given in the table is just an example.

Operations	**alu_operation (Controller Output)**
and	0000
or	0001
add	0010

Table 1.1:

Task 3: The next task is to decide the values of the **ALUOp** signal. In figure 1.3 observe that **ALUOp** does not depend on the instruction type only. Use this strategy to complete table 1.2 for all the instructions.

Task 4: Draw block diagram of first level controller which has one input **Opcode** and one output **ALUOp**. Determine their size yourself.

Instruction opcode	ALUOp	Operation	Funct7 field	Funct3 field	Desired ALU action	ALU control input
lw	00	load word	XXXXXXX	XXX	add	0010
sw	00	store word	XXXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXXX	XXX	subtract	0110
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001

Figure 1.3: ALUOp Table

Instruction	func3	func7	ALUOp	Desired ALU Action
and	000	0000000	10	AND
sub				
xor				
or				
and				
slt				
sll				
sltu				
srl				
sra				
addi				
xori				
ori				
andi				
slti				
slli				
sltui				
srli				
srai				
lb/sb				
lh/sh				
lw/sw				
lbu				
lhu				
lui				
auipc				
jal				
jalr				
beq				
bne				
blt				
bge				
bltu				
bgeu				

Table 1.2:

1.4 Deliverables

- Complete tables 1.1 and 1.2.
- Block diagrams of both controllers.

Acknowledgement

This manual is the result of the work of many including

1. Rehan Naeem (2026)
2. Tayyba Shafiq (2026)

Experiment 2

Implementation of the ALU and its Controller in SystemVerilog

2.1 Overview

In this experiment, we will design and implement key processor functional units in SystemVerilog. We will implement the ALU and its controller in system verilog. We will also test the functionality of these modules by writing testbenches in system verilog.

Additionally, we will also implement the Immediate Generator. This lab provides hands-on experience in writing combinational logic, control logic, and verifying hardware modules, forming the core building blocks of a processor datapath.

2.2 ALU Implementation in SystemVerilog

In this task, students will implement the Arithmetic Logic Unit (ALU) in SystemVerilog. A block diagram of the ALU is provided in Figure 2.1.

Task1: Implementation Write a System Verilog code to implement the ALU module which in turn implements the operations in Table 1.1. Input and output port names should be same as shown in the block diagram. Feel free to use additional flags when needed. For example, the **zero** flag is 1 when both operands are equal.

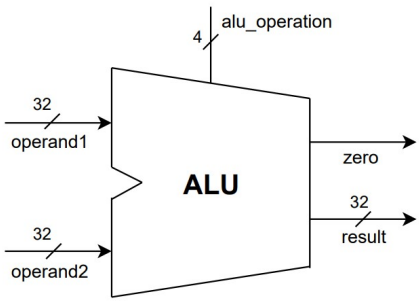

Figure 2.1: ALU

Task 2: Testing After implementing the ALU module, verify its functionality by writing a System Verilog randomized testbench. The testbench should apply a variety of input combinations and control signals to ensure that all arithmetic and logical operations produce the correct results.

2.3 ALU Controller Implementation

In this task, students will implement the ALU Controller in SystemVerilog.

Task 1: Implementation Implement the ALU Controller (second level controller) module provided in Figure 2.2 using the ALU operation table from Experiment 1 (see Table 1.2). Input/output port names should be same as shown in the block diagram.

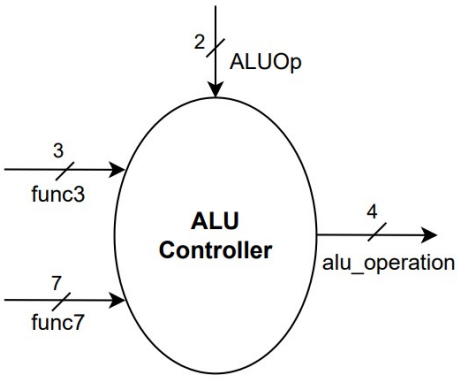

Figure 2.2: ALU Controller

Task 2: Write system verilog code for the first level controller which takes the opcode field of the instruction as input and outputs the ALUOp according to Table 1.2

Task 3: Testing Write a randomized testbench in SystemVerilog to verify the functionality of both the ALU controllers. The testbench should apply various instruction inputs and function codes to ensure that the controller generates the correct control signals for all ALU operations.

2.3.1 Use of `opcode.vh` File

While implementing the ALU controller in SystemVerilog, writing raw binary values for opcodes and function fields can easily lead to errors. To avoid such mistakes, students are provided with an opcode.vh header file containing predefined macros for all RISC-V opcodes and function codes.

Students are strongly encouraged to include this file in their SystemVerilog source files as shown below:

```
`include "opcode.vh"
```

After including the file, opcodes and function codes should be referenced using the defined macros instead of hard-coded binary values.

Example: To check for a load instruction, use:

```
if (opcode == `OPC_LOAD) begin
...
end
```

Similarly, other instructions can be identified using macros such as `` `OPC_BRANCH ``, `` `OPC_JAL ``, and `` `OPC_LUI ``. This approach improves code readability, reduces errors, and improves maintainability of the design.

2.4 Deliverables

- RTL and testbench codes for ALU and the two controllers.

Acknowledgement

This manual is the result of the work of many including

1. Rehan Naeem (2026)
2. Tayyba Shafiq (2026)

Experiment 3

Implementation of Immediate Generator in SystemVerilog

3.1 Overview

In this lab, we will design and implement an Immediate Generator (ImmGen) module in SystemVerilog for a RISC-V processor. The Immediate Generator extracts and sign-extends immediate values according to the RISC-V encoding rules.

We will study how to implement combinational logic to generate the correct 32-bit immediate output. The functionality of the module will be verified through simulation and testbench development.

3.2 Immediate Generator

The Immediate Generator (ImmGen) is a key combinational block in the RISC-V datapath. Its purpose is to extract the immediate field from a 32-bit instruction and generate a correctly formatted 32-bit immediate value that can be used by the ALU or other datapath components.

In RISC-V, different instruction formats (I-type, S-type, B-type, U-type, and J-type) encode immediate values using different bit fields of the instruction as observed from cheatsheet. Therefore, the Immediate Generator must extract the appropriate immediate bit fields, rearrange bits when required, perform sign-extension or zero-extension if required and produce a final 32-bit immediate value.

3.2.1 Block Diagram

The Immediate Generator takes a 32-bit **instruction** as input and produces a 32-bit **immediate** as output.

Task 1: Draw the block diagram of the Immediate Generator using draw.io. Label all signals properly and mention the bit-width of each input and output field.

3.2.2 Implementation

Task 2: Write a SystemVerilog module to implement the Immediate Generator, which takes a 32-bit **instruction** as input and produces a 32-bit **immediate** as output. The

module must support all instruction formats. Refer to the Riscv Cheat Sheet for immediate field decoding. Input/output port names should be the same as shown in Figure 3.1.

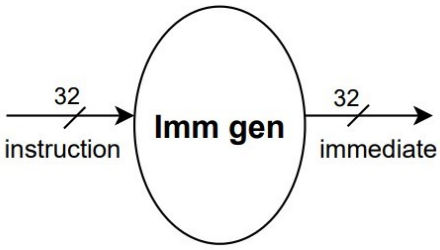

Figure 3.1: Immediate Generator Block Diagram

3.2.3 Testing

Task 3: Write a randomized testbench to verify the functionality of the Immediate Generator. Test the module using instructions from all supported formats and verify that the generated 32-bit immediate matches the expected value.

3.3 Deliverables

- Block diagram of Immediate Generator.
- RTL and testbench codes for Immediate Generator.

Acknowledgement

This manual is the result of the work of many including

1. Rehan Naeem (2026)
2. Tayyba Shafiq (2026)